

Logic Puzzles

Documentazione Tecnica del Progetto

Generatore e risolutore di Sudoku, KenKen e varianti tramite Z3 e backtracking Python.

Versione 0.2.0 · Python $\geq$ 3.10 · z3-solver $\geq$ 4.12

Web App: <https://sudokuandkenken.onrender.com>

1. Struttura del Progetto

Il progetto è organizzato come un package Python installabile (*puzzler*) con un modulo per ogni tipologia di puzzle. Un'interfaccia grafica autonoma (*gui.py*) permette di giocare interattivamente senza usare la CLI.

File / Cartella	Descrizione
gui.py	Interfaccia grafica tkinter (tema chiaro/scuro, timer, animazione di caricamento)
puzzler/main.py	Entry-point CLI — ArgumentParser con --game, --size, --difficulty, --seed
puzzler/core/	Utilità Z3 condivise: create_int_grid, model_to_grid, has_second_solution
puzzler/sudoku/	Sudoku classico 4×4 / 9×9 / 16×16
puzzler/kenken/	KenKen N×N ($N \geq 3$) con gabbie e operazioni aritmetiche
puzzler/diagonal/	Diagonal Sudoku — vincoli sulle due diagonali principali ($N = 4, 9$)
puzzler/hyper/	Hyper Sudoku — 4 finestre 3×3 aggiuntive ($N = 9$)
puzzler/nonconsecutive/	Nonconsecutive Sudoku — nessuna coppia adiacente con valori consecutivi ($N = 9$)
tests/test_smoke.py	22 test di smoke: unicità, riproducibilità con seed, correttezza vincoli
pyproject.toml	Dipendenze runtime (z3-solver) e dev (pytest, pytest-cov)
render.yaml	Configurazione deploy Render.com (build command, start command, Python version)
web/server.py	Backend FastAPI: endpoint /api/generate, routing ai generatori, serving file statici
web/requirements.txt	Dipendenze web: fastapi, uvicorn[standard], z3-solver
web/static/index.html	Single-page app: 3 schermate (home, loading, game) + win overlay
web/static/app.js	Logica di gioco completa: Canvas rendering, input touch/tastiera, timer, PWA SW
web/static/style.css	Layout mobile-first, tema chiaro/scuro con CSS custom properties
web/static/manifest.json	PWA manifest: nome, icona, display standalone, theme color
web/static/sw.js	Service worker: cache degli asset statici, bypass per /api/

2. Modulo Core — Utilità Z3

Il modulo **puzzler/core/solver_utils.py** espone tre funzioni condivise da tutti i moduli.

Funzione	Firma	Descrizione
create_int_grid	(name, n, lo, hi) → (X, constraints)	Crea griglia n×n di variabili Int Z3 con dominio [lo, hi] e restituisce variabili + vincoli di dominio

model_to_grid	$(\text{model}, X) \rightarrow \text{List}[\text{List}[\text{int}]]$	Traduce un modello Z3 in una griglia Python di interi leggibile
has_second_solution	$(s, X) \rightarrow (\text{bool}, \text{grid1}, \text{grid2})$	Verifica se esiste una seconda soluzione diversa dalla prima. Usa push/pop per non alterare il solver

2.1 has_second_solution — Tecnica push/pop

Per verificare l'unicità senza ricostruire il solver da zero, la funzione usa il meccanismo **push/pop** nativo di Z3:

- Chiama **s.check()** per trovare la prima soluzione.
- Crea un vincolo *diff_clause* = OR di tutte le celle diverse dalla prima soluzione.
- Esegue **s.push()**, aggiunge *diff_clause*, chiama **s.check()** di nuovo.
- Ripristina con **s.pop()**: il solver torna allo stato originale senza ricostruzione.

Questo approccio evita di ricostruire le clausole CDCL apprese dal SAT solver, riducendo il costo del secondo check rispetto a un solver fresco.

3. Sudoku Classico

Griglia $N \times N$ ($N \in \{4, 9, 16\}$) divisa in box $\sqrt{N} \times \sqrt{N}$. Ogni riga, colonna e box devono contenere tutti i valori da 1 a N esattamente una volta.

3.1 Modello Z3

La funzione **build_sudoku_model(n)** costruisce:

- N^2 variabili intere Z3 con dominio $[1, N]$.
- N vincoli **Distinct** sulle righe.
- N vincoli **Distinct** sulle colonne.
- N vincoli **Distinct** sui box $\sqrt{N} \times \sqrt{N}$ (in totale $3N$ vincoli **Distinct**).

Il numero totale di vincoli è **3N** Distinct, ognuno su N variabili.

3.2 Generazione della soluzione completa

Vengono usati due metodi in base alla dimensione della griglia:

Metodo	Usato per	Strategia	Costo
generate_full_solution	$N = 4, 9$	Ancora la prima riga con una permutazione casuale e lascia Z3 completare	< 0.5 s
generate_full_solution_backtracking	$N = 16$	Backtracking ricorsivo con euristica MRV + set incrementali per righe/colonne/box	< 1 s

3.3 Verifica dell'unicità

Il dispatcher **_check_unique** seleziona automaticamente il metodo:

- **$N \leq 9$** : Z3 con push/pop — ricostruisce il solver con gli indizi correnti e cerca una seconda soluzione.
- **$N = 16$** : backtracking Python (*_is_unique_backtracking*) — molto più veloce di Z3 su griglie grandi.

Backtracking iterativo per $N = 16$

L'algoritmo usa uno stack esplicito (no ricorsione) con:

- **MRV** (Minimum Remaining Values): sceglie sempre la cella con meno valori ammissibili.
- **Forward checking**: dopo ogni assegnazione aggiorna i domini dei peer, con backtrack immediato se un dominio si svuota.
- **Bitmask per i domini**: operazioni su insiemi in $O(1)$.
- **Snapshot** del dominio per ripristino $O(\text{celle vuote})$ al backtrack.
- **Time limit**: parametro *time_limit* (default 3 s per hard 16×16) — se esaurito restituisce None → la cella viene mantenuta (approccio conservativo).

3.4 Difficoltà e costi computazionali

Difficoltà	N=4	N=9	N=16	Celle visibili (N=9)
easy	< 1 s	< 2 s	< 5 s	~55% (~49)
medium	< 1 s	< 5 s	< 30 s	~35% (~28)
hard	< 2 s	< 30 s	< 60 s	minimale (~20-25)

3.5 Accorgimenti per ridurre il costo computazionale

- **Prima riga casuale:** ancorare la prima riga evita che Z3 esplori soluzioni simmetriche, dimezzando lo spazio di ricerca.
- **Dispatcher automatico:** per N=16 il backtracking Python è 10-50× più veloce di Z3 sia per la generazione che per l'unicità.
- **Time limit adattivo:** per N=16 medium/hard il check di unicità ha un budget temporale (0.5–3 s) per evitare esplosioni esponenziali.
- **Comportamento conservativo:** se il check va in timeout, la cella viene mantenuta — il puzzle risultante è più semplice ma sempre unico.
- **Percentuali adattive per N=16:** easy=55%, medium=47%, hard=43% (invece di 35%) per mantenere tempi di generazione ragionevoli.

4. KenKen

Griglia $N \times N$ ($N \geq 3$) senza box. Le celle sono raggruppate in **gabbie** (regioni connesse 4-adiacenti), ognuna con un'operazione aritmetica e un valore target. Righe e colonne devono contenere tutti i valori da 1 a N (latin square).

4.1 Operazioni supportate

Operatore	Simbolo	Celle	Vincolo Z3
Singleton	1	1	$X[r][c] == \text{target}$
Somma	+	≥ 2	$\text{Sum}(\text{celle}) == \text{target}$ (lineare)
Prodotto	*	≥ 2	Enumerazione tuple valide (non lineare $\rightarrow$ OR/AND)
Differenza	-	2	$\text{Or}(a-b==t, b-a==t)$ (lineare)
Divisione	/	2	$\text{Or}(a==t*b, b==t*a)$ (lineare)

Vincolo prodotto: enumerazione tuple

La moltiplicazione in Z3 è un'operazione **non lineare** e causa un'esplosione dei tempi di risoluzione. Per evitarlo, tutte le k -tuple $(v_1, \dots, v_k)$ con $v_i \in [1, N]$ il cui prodotto è uguale al target vengono pre-calcolate, e il vincolo viene espresso come:

```
Or([ And(v1==t1, v2==t2, ...) for (t1,t2,...) in valid_tuples ])
```

Questo converte l'operazione non lineare in una disgiunzione di vincoli lineari, gestibile efficientemente dal solver.

4.2 Generazione della soluzione (latin square)

Metodo	Usato per	Strategia	Costo
Z3 (prima riga casuale)	$N < 9$	Ancora la prima riga e lascia completare Z3	< 1 s
Ciclico + permutazioni	$N \geq 9$	$L[i][j] = (i+j) \% N + 1$, poi permutazioni casuali di righe, colonne e simboli — $O(N^2)$	< 0.01 s

4.3 Partizionamento BFS con garanzia di connettività

La funzione `_partition_into_cages` usa BFS random per creare le gabbie:

- Per ogni cella non assegnata, sceglie un seed casuale.
- Espande la gabbia aggiungendo vicini dal frontier solo se adiacenti alla gabbia in costruzione (`_is_adjacent_to_set`).
- Garantisce che ogni gabbia formi un **polinomio connesso** (senza buchi).
- La dimensione massima della gabbia è limitata da `max_cage_size`.
- `singleton_rate` controlla la probabilità che una cella diventi gabbia singleton direttamente.

4.4 Garanzia di unicità — solver incrementale

La funzione `_ensure_unique` garantisce unicità con un approccio incrementale:

- Costruisce il solver Z3 **una sola volta** con tutti i vincoli delle gabbie.
- Chiama `has_second_solution` per trovare soluzioni alternative.
- Se il puzzle non è unico, sceglie preferibilmente celle dove la soluzione alternativa **differisce** dalla nostra (ogni pin elimina quella soluzione).
- Aggiunge vincoli di singleton ($X[r][c] == val$) direttamente al solver senza ricostruirlo — Z3 riusa le clausole CDCL apprese (**warm start**).
- Ricostruisce le gabbie rimanenti mantenendo la connettività tramite `_connected_components`.

4.5 Modalità hard — fusione greedy

La funzione `generate_minimal_kenken_puzzle` genera il puzzle con minima informazione:

- Parte da N^2 singleton (massima informazione — ogni cella è nota).
- Prova a fondere coppie adiacenti in ordine casuale.
- Accetta la fusione solo se la soluzione rimane unica (verifica con `has_second_solution`).
- Ripete finché nessuna ulteriore fusione è possibile.

4.6 Costi computazionali KenKen

Dimensione	easy	medium	hard (minimal)
3×3	< 1 s	< 1 s	< 2 s
4×4	< 1 s	< 1 s	< 3 s
6×6	< 1 s	< 2 s	< 5 s
9×9	< 2 s	~3 s	< 15 s

5. Varianti Sudoku

5.1 Diagonal Sudoku

Estende il Sudoku classico con due vincoli aggiuntivi:

- Diagonale principale $(0,0) \rightarrow (N-1,N-1)$: tutti i valori da 1 a N distinti.
- Diagonale secondaria $(0,N-1) \rightarrow (N-1,0)$: tutti i valori da 1 a N distinti.

Dimensioni supportate: **N = 4** e **N = 9**. I due vincoli Distinct extra riducono lo spazio di ricerca di Z3, rendendo la generazione più veloce rispetto al Sudoku classico.

La generazione usa la stessa pipeline del Sudoku classico (Z3 per soluzione + Z3 per unicità). Tempi: easy < 2 s, medium < 5 s, hard < 60 s per N=9.

5.2 Hyper Sudoku

Estende il Sudoku classico 9×9 con 4 regioni 3×3 aggiuntive (finestre) sfalsate rispetto ai box standard:

- Finestra 1: righe 1-3, colonne 1-3 (top-left)
- Finestra 2: righe 1-3, colonne 5-7 (top-right)
- Finestra 3: righe 5-7, colonne 1-3 (bottom-left)
- Finestra 4: righe 5-7, colonne 5-7 (bottom-right)

Il modello aggiunge 4 vincoli **Distinct** sulle 9 celle di ciascuna finestra. I vincoli extra rendono il puzzle più vincolato, permettendo meno indizi per garantire l'unicità. Supporto: **N = 9 solamente**.

Tempi: easy 10-30 s, medium 30-60 s. La generazione è più lenta del Sudoku classico perché i vincoli aggiuntivi rendono il problema più difficile da soddisfare per Z3.

5.3 Nonconsecutive Sudoku

Estende il Sudoku classico 9×9 con il vincolo che nessuna coppia di celle ortogonalmente adiacenti possa avere valori consecutivi:

```
|a - b| != 1    per ogni coppia (a, b) di celle adiacenti
```

Implementato come due vincoli lineari separati per ogni coppia adiacente:

```
x[r][c] - x[r][c+1] != 1
x[r][c+1] - x[r][c] != 1
```

Nota: N=4 è UNSAT — con valori 1-4 le uniche righe valide sono [2,4,1,3] e [3,1,4,2], che risultano sempre consecutive in verticale. Supporto: **N = 9 solamente**.

Ottimizzazione: Z3 incrementale con assumptions

La generazione del Nonconsecutive usa la tecnica più avanzata del progetto:

- Il modello Z3 viene costruito **una sola volta** senza indizi.
- Le clue del puzzle corrente vengono passate come **assumptions temporanee** ad ogni check di unicità.

- Z3 non aggiunge le assumptions permanentemente: le **clausole CDCL apprese** rimangono tra una chiamata e l'altra (warm start), rendendo ogni check 5-10× più veloce rispetto al rebuild.
- Per la modalità hard viene impostato un **timeout di 2 s** per check: se Z3 non riesce a dimostrare l'unicità entro il budget, la cella viene mantenuta (approccio conservativo).
- Le espressioni Z3 per le clue ($clue_expr[r][c] = X[r][c] == val$) vengono **pre-create una volta** prima del ciclo, risparmiando il costo di costruzione delle espressioni.

Difficoltà	Tempo	Indizi (~)	Note
easy	~25 s	~45	~55% celle visibili
medium	~25 s	~28	~35% celle visibili
hard	< 90 s	~15-22	Minimale con timeout 2 s/check — puzzle unico e vicino al minimale

6. Utilizzi di Z3 nel Progetto

Z3 (Microsoft Research) è un *SMT solver* (Satisfiability Modulo Theories) usato come motore di constraint satisfaction. Nel progetto viene impiegato in cinque modi distinti:

Utilizzo	Dove	Tecnica Z3
Generazione soluzione	Sudoku $N \leq 9$, KenKen $N < 9$, Diagonal, Hyper, Nonconsecutive	Solver standard: vincoli Distinct + dominio, prima riga casuale come ancora
Risoluzione puzzle	Tutti i moduli (funzioni solve_*)	Solver con clue aggiunte come vincoli di uguaglianza — verifica SAT/UNSAT
Verifica unicità (push/pop)	has_second_solution in core	push() → aggiunge diff_clause → check() → pop() per ripristinare stato
Unicità incrementale (KenKen)	_ensure_unique in kenken/model.py	Solver condiviso, singleton aggiunti in append-only — Z3 riusa clause CDCL (warm start)
Unicità con assumptions (Nonconsecutive)	_is_unique_assumptions in nonconsecutive/model.py	s.check(*assumptions): clue temporanee senza modificare il solver — warm start CDCL tra iterazioni

6.1 Confronto delle tecniche Z3

Le tre tecniche per la verifica dell'unicità si differenziano per costo e applicabilità:

Tecnica	Costo relativo	Uso principale	Limitazione
Rebuild completo	Alto (baseline)	Verifica esterna / test	Nessun riuso clause
push/pop	Medio (-30%)	Sudoku, Diagonal, Hyper	Riuso parziale
Incrementale append-only	Basso (-50%)	KenKen _ensure_unique	Solo aggiunta vincoli
Assumptions (temporanee)	Minimo (-70%)	Nonconsecutive (ciclo rimozione)	Solo vincoli temporanei

6.2 Quando Z3 viene sostituito

In due scenari Z3 è troppo lento e viene sostituito da algoritmi Python puri:

- **Sudoku 16×16 — generazione:** Z3 impiega minuti. Il backtracking con MRV termina in < 1 s.
- **Sudoku 16×16 — unicità:** Z3 non scala. Il backtracking iterativo con bitmask e forward checking è 10-50× più veloce.
- **KenKen $N \geq 9$ — latin square:** Z3 impiega > 20 s. La costruzione ciclica $O(N^2)$ termina in < 0.01 s.

7. Riepilogo degli Accorgimenti Computazionali

Accorgimento	Modulo	Beneficio
Prima riga casuale come ancora	Sudoku, KenKen, Nonconsecutive	Guida Z3 verso soluzioni diverse senza aumentare i vincoli; riproducibilità con seed
Backtracking MRV (Minimum Remaining Values)	Sudoku 16×16	Riduce i backtrack scegliendo sempre la cella con meno candidati; da minuti a < 1 s
Bitmask per i domini (forward checking)	Sudoku 16×16	Operazioni su insiemi $O(1)$ invece di $O(N)$; snapshot $O(\text{celle vuote})$ per backtrack
Latin square ciclico $O(N^2)$	KenKen $N \geq 9$	Elimina Z3 dalla generazione del latin square: > 20 s → < 0.01 s
Enumerazione tuple per prodotto	KenKen	Converte moltiplicazione non lineare in OR/AND lineare — evita esplosione del solver
Soglia prodotto n^k	KenKen _make_cage	Evita target enormi nelle gabbie * che rallentano l'enumerazione delle tuple valide
Solver incrementale (append-only)	KenKen _ensure_unique	Riusa clausole CDCL tra i check di unicità; ogni pin elimina immediatamente la soluzione alternativa
Assumptions temporanee (CDCL warm start)	Nonconsecutive	Il solver accumula conoscenza tra iterazioni senza invalidarla; 5-10× più veloce del rebuild
Pre-creazione espressioni Z3	Nonconsecutive	Le clue_expr vengono create una volta prima del ciclo, eliminando overhead di costruzione
Time limit adattivo per check	Sudoku 16×16, Nonconsecutive hard	Limita il tempo per singolo check di unicità; approccio conservativo per puzzle sempre unici
Dispatcher automatico N	Sudoku, KenKen	Seleziona automaticamente Z3 o backtracking in base alla dimensione senza intervento utente
Fusione greedy (hard KenKen)	generate_minimal_kenken_puzzle	Parte dal massimo di informazione (singleton) e rimuove progressivamente — garantisce minimalità

8. Interfaccia Grafica (gui.py)

L'interfaccia grafica è realizzata con **tkinter** (libreria standard Python) e permette di giocare a tutti i puzzle senza usare la CLI.

8.1 Avvio

```
python gui.py
```

8.2 Schermate principali

- **Home:** selezione di gioco, dimensione e difficoltà tramite pulsanti toggle. La lista delle dimensioni si aggiorna automaticamente in base al gioco scelto.
- **Loading:** animazione spinner 3×3 (comet rotante) mentre la generazione avviene in un thread in background — la UI rimane responsive. Al termine mostra il tempo di generazione.
- **Gioco:** griglia disegnata su Canvas tkinter con sfondo, bordi e numeri. Timer live, pulsante Solve e navigazione da tastiera (freccie, backspace).
- **Win dialog:** modale con tempo impiegato, nome del gioco e difficoltà, pulsante New Game.

8.3 Funzionalità dell'area di gioco

Funzionalità	Descrizione
Canvas-based rendering	Griglia disegnata con rettangoli e testo su Canvas — nessun widget Entry, controllo totale su stile e bordi
Bordi adattivi	Sudoku: linee spesse ogni $\sqrt{N}$ celle (box). KenKen: linee spesse ai confini delle gabbie. Hyper: bordi amber sulle 4 finestre
Tinte per varianti	Diagonal: celle diagonali in giallo. Hyper: finestre in amber. Tinte integrate nella logica del background, non sovrascrivibili
Input multi-cifra	Per N=16: buffer con timeout 600 ms — digitare '1' poi '6' inserisce 16
Feedback errori	Celle sbagliate in rosso in tempo reale. Celle risolte tramite pulsante in verde
Tema chiaro/scuro	Toggle persistente (icona luna/sole). Al cambio tema lo stato di gioco (griglia, timer, flag solve/win) viene preservato
Generazione asincrona	Thread daemon in background — la finestra non si blocca durante la generazione
Dimensione finestra stabile	Durante la generazione la finestra mantiene le dimensioni della schermata Home

9. Web App / Progressive Web App

Il progetto include una **web app mobile-first** accessibile all'indirizzo <https://sudokuandkenken.onrender.com> e installabile come applicazione nativa su Android e iOS tramite il meccanismo PWA.

9.1 Architettura

L'app è strutturata in due livelli distinti:

- **Backend (web/server.py):** server FastAPI che espone un singolo endpoint REST POST `/api/generate`. Riceve `game`, `n`, `difficulty` e `seed` opzionale; istanzia il generatore Python corrispondente; restituisce `puzzle`, `solution`, `cages`, `elapsed` e `seed` in JSON.
- **Frontend (web/static/):** single-page app HTML/CSS/JS pura, senza framework. Usa Canvas 2D API per il rendering della griglia, identico nella logica alla versione tkinter.

Campo richiesta	Tipo	Descrizione
game	string	sudoku kenken diagonal hyper nonconsecutive
n	int	Dimensione griglia (es. 4, 9, 16)
difficulty	string	easy medium hard
seed	int?	Seed opzionale per riproducibilità (default: casuale)

9.2 Frontend — Schermate e Canvas

- **Home:** toggle per gioco, dimensione e difficoltà. La lista dimensioni si aggiorna dinamicamente in base al gioco selezionato.
- **Loading:** animazione 9 punti 3×3 (comet rotante) con fetch asincrono a `/api/generate`. Al completamento mostra il tempo di generazione.
- **Game:** griglia su Canvas HTML5 con DPR-aware rendering (`canvas.width = size * devicePixelRatio + ctx.setTransform` per display Retina). Numpad touch per mobile (pulsanti 1-N + cancella). Input multi-cifra da tastiera con buffer 600 ms per N=16. Timer live, pulsante Solve, navigazione frecce.
- **Win overlay:** modale con tempo impiegato e dati del puzzle. Pulsante New Game torna alla home.

9.3 Rendering Canvas — Corrispondenza con tkinter

Il rendering web rispecchia fedelmente la GUI tkinter:

Elemento visivo	tktinter	Canvas HTML5
Sfondo celle	<code>_draw_cell_bg()</code>	<code>cellBgColor()</code> + <code>fillCell()</code>
Linee griglia	<code>_draw_lines()</code>	<code>drawLines()</code>
Bordi gabbie KenKen	<code>_draw_cage_borders()</code>	<code>drawCageBorders()</code>
Bordi Hyper (amber)	<code>_draw_hyper_borders()</code>	<code>drawHyperBorders()</code>

Label gabbie	<code>_draw_cage_labels()</code>	<code>drawCageLabels()</code>
Valori celle	<code>_draw_values()</code>	<code>drawValue()</code>
Selezione cella	Rettangolo accent	<code>strokeRect</code> accent
Tinta diagonale	diag_tint giallo	<code>C.diag_tint</code> giallo
Tinta Hyper	hyper_tint amber	<code>C.hyper_tint</code> amber

9.4 Progressive Web App (PWA)

- **manifest.json:** definisce nome, icona SVG inline, display standalone, theme_color. Permette l'installazione come app nativa su Android (Chrome) e iOS (Safari).
- **sw.js (Service Worker):** intercetta le richieste in fetch. Mette in cache '/', '/static/style.css', '/static/app.js', '/manifest.json' all'installazione. Bypassa la cache per tutte le chiamate a /api/ (sempre fresh dal server).
- **DPR handling:** `canvas.width = logicalSize * devicePixelRatio`; `ctx.setTransform(dpr, 0, 0, dpr, 0, 0)`. Garantisce rendering nitido su display Retina e HiDPI mobile.

9.5 Deploy su Render.com

L'app è deployata gratuitamente su Render.com con integrazione GitHub automatica:

- **Build command:** `pip install -r web/requirements.txt`
- **Start command:** `bash -c 'cd web && exec uvicorn server:app --host 0.0.0.0 --port $PORT'`
- **Python version:** 3.11 (pinned via `env var PYTHON_VERSION`).
- **Deploy automatico:** ogni push su main triggera un nuovo deploy senza intervento manuale.
- **Limitazione piano gratuito:** il servizio va in sleep dopo 15 minuti di inattività; il primo wake-up impiega ~30-50 secondi.

Funzionalità	Dettaglio
Tutti i 5 giochi	Sudoku, KenKen, Diagonal, Hyper, Nonconsecutive — stessa logica del backend Python
Tema chiaro/scuro	CSS custom properties (<code>:root / [data-theme=dark]</code>); palette identica a tkinter
Responsive mobile	<code>canvasLogicalSize() = min(viewport - 32, 560)</code> ; adatta il canvas alla finestra
Input touch	touchstart su canvas con <code>preventDefault</code> ; numpad con pulsanti touch-friendly
Input tastiera 16×16	Buffer multi-cifra con timeout 600 ms — digita '1' poi '6' per inserire 16
Installabile come app	PWA: manifest + service worker; Add to Home Screen su Android/iOS
Cache offline	Service worker mette in cache gli asset statici; API sempre online

10. Tabella Riepilogativa dei Costi

Puzzle	Dimension i	Generaz. soluzione	Unicità	easy	medium	hard
Sudoku	4, 9, 16	Z3 (4,9) / BT (16)	Z3 (≤ 9) / BT (16)	<2s	<5s	<60s
KenKen	3-9	Z3 (<9) / Ciclico (≥ 9)	Z3 incrementale	<2s	~3s	<15s
Diagonal Sudoku	4, 9	Z3	Z3	<2s	<5s	<60s
Hyper Sudoku	9	Z3	Z3	10-30s	30-60s	—
Nonconsecutive	9	Z3	Z3 assumptions	~25s	~25s	<90s

BT = Backtracking Python con MRV e forward checking. I tempi sono indicativi e dipendono dall'hardware.